CSCI 551 – Numerical and Parallel Programming: Exercise #5

Linear Systems and Series with MPI and OpenMP

DUE: As indicated on Canvas

Please read Week-10 and Week-11 notes and bring questions to activity. You should expect to spend about 1 hour per 10 points, and complete spending 5 hours per week.

You will need an ECC system (***ecc-linux.csuchico.edu***) account or access to a native Linux system that is multi-core to work on this assignment. Most students can just SSH to the ECC system using MobaXterm, Cyberduck, or Putty or whatever your favorite SSH tool is, but please obtain an account one if you don't already have (https://www.csuchico.edu/itss/index.shtml).

Note that you can clone the tested starter code from https://github.com/sbsiewertcsu/numeric-parallel-starter-code (recommended) or download the starter code from the class web-site one file at a time at http://www.ecst.csuchico.edu/~sbsiewert/csci551/code/, which is "sandbox" code. ***Github is the best option – use "git clone https://github.com/sbsiewertcsu/numeric-parallel-starter-code.git" to make a copy on ECC-Linux.***

For analysis and design work, you will find the Pacheco book MPI companion materials useful as well as use of the online Kaw and Kalu numerical reference book for methods of numerical integration, interpolation, and sources of error in numerical computation.

All code submissions must be accompanied by a Makefile or similar build automation with documentation on how to build with a "single command" from the command line on the ECC server (***ecc-linux.csuchico.edu***) where MPI is available with the Intel Parallel Studio XSE installation. Report your results, clearly answering each of the questions below with analysis, example output, code snippets, and/or mathematical analysis and solutions (answers clearly indicated by boxing them).

ECC Cluster node use policies – Based on your birthday and year, if your year of birth is even, use "o244" nodes and if you birth year is odd, use "o251" nodes. For POSIX shared memory threading (single node use), login to the node # that is the same as your birthday – e.g., for me, odd year, 14th day of month, so I would use "ssh o251-14". This should help distribute the load as we get into problems that are more CPU, I/O, and memory intensive.

Exercise #5 Objectives:

For the following four problems stress linear systems using MPI and introduce OpenMP with a simple series approximation that we have previously seen and implemented with MPI. The learning objectives are:

- Review linear systems and vector/matrix notation for them along with methods to solve them. Learn how to solve them by hand and with automation that you code, but also how to check your work using math solver tools (e.g. https://www.symbolab.com/, Handy Math, OnlineMSchool, Sage Math, and MATLAB Online or CSU MATLAB). ***Note that you must do work requested as original work, but you are encouraged to use tools to check and verify your work.***
- Use resultant solution vectors from linear problems and multiply by original coefficient matrix to verify that you can regenerate the original problem RHS (Right Hand Side) without significant error.
- Compare OpenMP (shared memory parallel) to MPI (distributed memory parallel) for a simple series approximation problem. We will work with OpenMP more in our final assignment.
- Use MPI to implement simple vector/matrix operations (addition, subtraction, multiplication, and determination of properties) based upon examples and custom code as a review.
- Solve a challenging real-world linear systems problem using one of three standard approaches (Gaussian Elimination with Partial Pivoting, Gauss Jordan, LU factorization, or Gauss Seidel iteration), first with starter code, and then with code you have adapted to speed-up with MPI (and/or OpenMP) parallel code design methods.

1) [20 points total] The three main methods to solve a 3x3 system of linear equations include: simple substitution, Cramer's rule, and elimination with augmented matrix using row operations. Compare hand solving with automated row operations (e.g., Gaussian Elimination with partial pivoting, Gauss Jordan, or LU factorization) and with iterative methods (Gauss Seidel). Solve the following equations:

$$3x - 0.1y - 0.2z = 7.85$$
$$0.1x + 7y - 0.3z = -19.3$$
$$0.3x - 0.2y + 10z = 71.4$$

[5 pts] By substitution or Cramer's rule, showing work, with verification (***e.g, symbolab***)
[5 pts] By elimination with an augmented matrix, showing work, with verification (***hint: use gewpp.c with DEBUG and TRACE on to help you with steps and watch video help***)
[5 pts] Use a C/C++ program to automate row operations (e.g. gewpp.c) and note any differences from your solution in terms of time, accuracy, and precision.
[5 pts] Use a C/C++ program to automate iteration (e.g. gsit.c) and note any differences from your solution in terms of convergence, time, accuracy, and precision. Be sure to organize equations in diagonally dominant form.

Comment on which program you believe works best or if they are all equally good.

2) [30 points total] Timing MPI and CUDA programs. The goal of this exercise is to practice methods to determine speed-up (SU), to compute P% (using formula), and to assess problem scaling using either "htop" for CPU or "nvtop" for GP-GPU usage monitoring.

   [20 pts] Test, explain (as best you can), and assess the problem scaling and GP-GPU P% scaling for "transpose" (scaling X, Y dimensions for square matrices) ***using CUDA Sample transpose code***. Note that if you do not clone or pull the latest code from the numeric-parallel github, you may also need header files in cuda_common. Time the transpose for a range of matrix sizes (256, 512, 1024, 4096, 8192) and plot GP-GPU use and P% (plot or summarize in a table) and show the peak GP-GPU use you see with nvtop. If you can't compute P%, estimate it based on observation of GP-GPU loading. Note that you can also compare the CUDA to sequential transpose.c code to compute SU as well (same math, but different code), but the focus here is problem scaling and how much of the work is "off-loaded" from the CPU and handled by the GP-GPU rather than CPU cores.

   [10 pts] Re-implement this MPI program (piseriesreduce.c) that estimates the value of Π by two well-known series noted in the MPI code, now with OpenMP (see pp. 229 in Pacheco) and show that you can run both sequentially and parallel to compare both. Using MPI_Wtime and the POSIX clock_gettime, compare the two implementations for 2 or more workers (# of ranks >= 2) for 100,000 up to 1,000,000 iterations in terms of speed and note any differences in scaling, accuracy, or precision between OpenMP and MPI for the series.

3) [20 points total] Vector matrix operations can become time consuming when scaled to large dimensions. Implement the following operations as a simple C/C++ sequential program and then re-implement as an OpenMP, MPI or CUDA program to determine if you can achieve parallel speed-up for the operation.

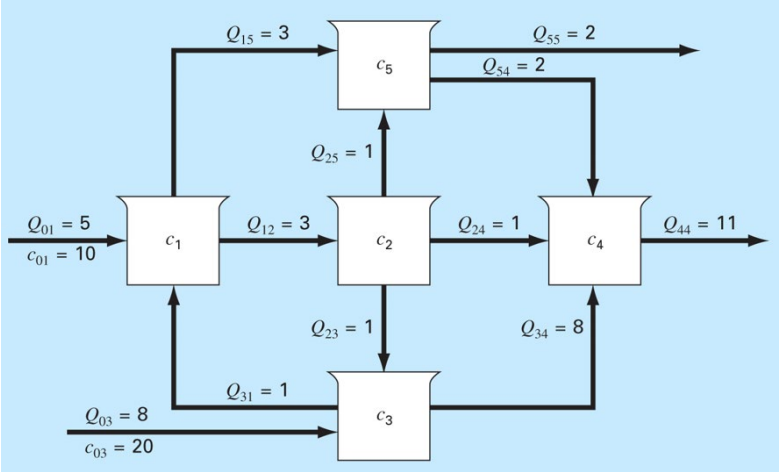
   [10 pts] Multiply any dimension square matrix from 2x2 up to NxN by another matrix of the same dimensions, or by a column vector of dimension 1 up to N. Make your program interactive and so that you can test it by re-directing input to it for faster test automation.

   [10 pts] Create an OpenMP or MPI version of your program above (your pick) that allows for multiple worker parallelism to speed up the square matrix multiplication above as well as the matrix and column vector multiplication. ***Test it for functional correctness with a 3x3 matrix and dimension 3 vectors for simplicity. However, please also test it for scaling with a 10x10, 100x100 and larger NxN for speed-up achieved with your parallel implementation. Report on the speed-up achieved for 3x3, 10x10, 100x100, and even 1000x1000 if this is feasible.***

   For both, provide an example that uses the following test matrix and vector for verification:

$$m1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, m2 = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}, v1 = \begin{bmatrix} 11 \\ 7 \\ 13 \end{bmatrix},$$

4) [30 points total] The following problem uses linear systems to model chemical reactor vessels that are linked by pipes as shown below [1].



The concentration of a dissolved chemical "c" and the mass flow rate "Q" can be expressed in a system of linear equations based upon conservation of mass. You simply need to solve the linear system of equations, but if you are interested in how these 5 equations odel the system see Ex-5-Chemical-Mixing-Problem-Background-on-Linear-System-Model.pdf. From Figure 1, found in ***Numerical methods for engineers***, we can derive a full set of consistent simultaneous linear equations as follows (note that Q drops out because inflow must equal outflow):

Eq1: $6c1 - c3 = 50$
Eq2: $-3c1 + 3c2 = 0$
Eq3: $-c2 + 9c3 = 160$
Eq4: $-c2 - 8c3 + 11c4 - 2c5 = 0$
Eq5: $-3c1 - c2 + 4c5 = 0$

[10 pts] Solve for the 5 unknown concentrations in the system c1 … c5 with a ***sequential program***.

[20 pts] Solve for the 5 unknown concentrations in the system c1 … c5 with an ***MPI or OpenMP*** parallel program. Show that the program is equivalent and measure execution with "time" on the command line to determine if it runs any faster and if you see not speed-up with time, try measuring the time for just the parallel section using POSIX clock_gettime.

You can ***verify your programs*** with MATLAB or symbolab or other math solvers if you wish, but ***do not use programs to generate work you have been asked to try by hand*** unless you get stuck or hand solving is taking way longer than expected for the problem points. You can also use a numerical method you are asked to code in C/C++, use, or adapt from existing C/C++ with verification. Make sure that you show you have tested your programs and provide sample build and run instructions and/or screen shots.

References:

[1] Chapra, Steven C., and Raymond P. Canale. *Numerical methods for engineers*. Boston: McGraw-Hill Higher Education,7th Edition, 2015. (Ex-5-Chemical-Mixing-Problem-Background-on-Linear-System-Model.pdf)

Overall, provide a well-documented professional report of your findings, output, and tests so that it is easy for a colleague (or instructor) to understand what you have done, what worked, what did not and why (even if you cannot complete to your satisfaction). Provide clear instructions on how to run your programs, including command line arguments required and screenshots demonstrating use and test cases you used to verify your parallel and sequential programs.

Include any design files or log files, C/C++ source code you write (or modify) and Makefiles needed to build your code. I will look at your report first, so it must be well written and clearly address each problem providing clear and concise responses and example results (e.g. summary analysis and clearly boxed mathematical answers) to receive credit, but I will look at your log files, code and test results as well if I have questions.

Report file MUST be separate from the ZIP file with code and other supporting materials.

Rubric for Scoring for scale 0…10

Score	Description of reporting and code quality
0	No answer, no work done
1	Attempted and some work provided, incomplete, does not build, no Makefile
2	Attempted and partial work provided, but unclear, Makefile, but builds and runs with errors
3	Attempted and some work provided, but unclear, build warnings, runs with no apparent error, but not correct or does not terminate
4	Attempted and more work provided, but unclear, build warnings, runs with no apparent error, but not correct or does not terminate
5	Attempted and most work provided, but unclear, build warnings, runs with no apparent error, but not correct or does not terminate
6	Complete answer, but does not answer question well and code build and run has warnings and does not provide expected results
7	Complete, mostly correct, average answer to questions, with code that builds and runs with average code quality and overall answer clarity
8	Good, easy to understand and clear answer to questions, with easy-to-read code that builds and runs with no warnings (or errors), completes without error, and provides a credible result
9	Great, easy to understand and insightful answer to questions, with easy-to-read code that builds and runs cleanly, completes without error, and provides an excellent result
10	Most complete and correct - best answer and code given in the current class

Grading Checklist for Rubric

[20 points] Hand solution of a simple 3x3 linear system problem compared to automated:

Problem	Possible	Score	Comments
Method #1	5		
Method #2	5		
Program #1	5		
Program #2	5		
TOTAL			

[30 points] OpenMP compared to MPI for series approximations:

Problem	Possible	Score	Comments
Analyze CPU offload to GP-GPU for CUDA Sample code "transform"	20		
Compare MPI code to sequential and determine speed-up with timestamps	5		
Compare OpenMP code to sequential and determine speed-up with timestamps	5		
TOTAL			

[20 points] Review of Vector/Matrix operations using MPI:

Problem	Possible	Score	Comments
Multiply any dimension square matrix from 2x2 up to NxN by another matrix	10		
Create an OpenMP or MPI version of your program above – Test it for speed-up and verify	10		
TOTAL			

[30 points] Linear system real-world problem sequential and parallel solution:

Problem	Possible	Score	Comments
Selection of solution algorithm (Gaussian Elimination with PP, Gauss Jordan, LU factorization, or Gauss Seidel iteration)	5		
Sequential solution & verification	5		
Parallel program design and test	10		
Parallel program solution sequential time comparison	10		
TOTAL			